D532 Final Project Report

Joseph Malone

Part 1: Application Content

1.1 Raw Content

There are no CSV or Excel files in this project. Raw data comes from the **Chess.com public API**, which provides monthly game archives as JSON. Each entry in the archive contains an embedded PGN string--the standard format for recording chess games. The ETL pipeline fetches these archives on demand and parses the PGN directly in memory.

An example PGN record looks like this:

```
[Event "Live Chess"]
[Site "Chess.com"]
[Date "2026.04.12"]
[White "jpmalone"]
[Black "opponent123"]
[Result "1-0"]
[WhiteElo "1901"]
[BlackElo "1888"]
[TimeControl "600"]
[ECO "C60"]
[ECOUrl "https://www.chess.com/openings/Ruy-Lopez-Opening"]
[Termination "jpmalone won by resignation"]

1. e4 {[%clk 0:10:00]} 1... e5 {[%clk 0:9:59]} 2. Nf3 {[%clk 0:09:59]} ...
```

The `[%clk ...]` annotations embedded in move comments are parsed to extract per-move clock times, which allows us to create features surrounding clock times like the clock advantage and move time distribution features.

The Chess.com API endpoint used is:

```
GET https://api.chess.com/pub/player/{username}/games/{YYYY}/{MM}
```

1.2 Database Content

The application uses a **SQLite** database (`chess_analytics.db`) with three tables. The database acts as a persistent local cache—once a game is imported, it is never re-fetched (deduplication via `chess_com_url`).

Schema

```sql
CREATE TABLE players (
    player_id   INTEGER PRIMARY KEY,
    username    VARCHAR(100) NOT NULL UNIQUE,
    platform    VARCHAR(20) NOT NULL DEFAULT 'chess.com'
);

CREATE TABLE games (
    game_id         INTEGER PRIMARY KEY,
    white_player_id INTEGER NOT NULL REFERENCES players(player_id),
    black_player_id INTEGER NOT NULL REFERENCES players(player_id),
    result          VARCHAR(10) NOT NULL,       -- '1-0', '0-1', '1/2-1/2'
    date_played     DATE,
    time_control    VARCHAR(30),                -- e.g. '600', '180+2'
    time_class      VARCHAR(20),                -- bullet, blitz, rapid,
daily
    white_elo       INTEGER,
    black_elo       INTEGER,
    white_accuracy  FLOAT,                      -- from chess.com
(nullable)
    black_accuracy  FLOAT,
    eco             VARCHAR(10),                -- ECO opening code
    opening_name    VARCHAR(255),
    termination     VARCHAR(255),
    chess_com_url   VARCHAR(255) UNIQUE,        -- deduplication key
    total_moves     INTEGER,
    created_at      DATETIME
);

CREATE TABLE moves (
    move_id            INTEGER PRIMARY KEY,
    game_id            INTEGER NOT NULL REFERENCES games(game_id) ON DELETE
CASCADE,
    ply                INTEGER NOT NULL,         -- 1-indexed half-move
    move_number        INTEGER NOT NULL,
    color              VARCHAR(5) NOT NULL,      -- 'white' or 'black'
    move_san           VARCHAR(10) NOT NULL,
    clock_seconds      FLOAT,                   -- remaining clock after
this move
    time_spent_seconds FLOAT,                   -- time consumed on this
move
    UNIQUE (game_id, ply)
);
```

Performance indices are defined on `games(white_player_id)`, `games(black_player_id)`, `games(date_played)`, `games(time_class)`, `moves(game_id, ply)`, and `moves(game_id, color)`.

The following shows a few queries including the number of entries for each table and example records for each. The data is stored on my local device accumulated by searching for players and pulling their associated records.

1.3 Application Interface Code

The application is a FastAPI backend with a single-page HTML/JS frontend built on Chart.js.

Since this was a solo project and I have limited JavaScript and HTML experience, I used Claude Code to help develop and structure the frontend once the backend and database were working. The backend, ETL pipeline, data models, and all analytics queries were written independently.

The backend exposes a REST API (`app/main.py`) that the frontend calls via `fetch`. For example:

```
GET /api/players/{username}/analytics/elo-history?
time_class=rapid&start_date=2025-01-01
GET /api/players/{username}/analytics/clock-advantage?time_class=rapid
GET /api/players/{username}/sync  [POST — triggers Chess.com import]
```

The frontend (`app/static/app.js`) renders interactive Chart.js visualizations in response. It includes a **compare mode** that loads two players side by side and syncs the Y-axes of corresponding charts.

A few screenshots showing the dashboard can be found below.

1.4 Interaction (Queries)

All analytics are implemented in `app/crud.py` as explicit SQL strings executed via `sqlalchemy.text()`. SQLAlchemy is used as a database connection layer. Below are a few examples.

Elo history

Fetches each player's Elo rating over time using a `CASE WHEN` expression to select the correct Elo column depending on whether the player was white or black:

```
-- app/crud.py, elo_history()
SELECT
    g.date_played,
    CASE WHEN g.white_player_id = :player_id
         THEN g.white_elo ELSE g.black_elo
    END AS elo,
    g.time_class
FROM games g
WHERE (g.white_player_id = :player_id OR g.black_player_id = :player_id)
  AND g.date_played IS NOT NULL
```

```
  AND CASE WHEN g.white_player_id = :player_id
           THEN g.white_elo ELSE g.black_elo
      END IS NOT NULL
ORDER BY g.date_played ASC, g.game_id ASC
```

After the query, an IQR-based outlier filter removes anomalous spikes caused by inactivity periods pulling in games from a different time control.

Player stats with common table expression and per-color move counts

Uses a CTE to compute outcomes via `CASE WHEN`, then joins to `moves` filtered to only the player's own half-moves (not their opponent's):

```
-- app/crud.py, get_player_stats()
WITH player_games AS (
    SELECT
        g.game_id, g.result, g.time_class,
        CASE WHEN g.white_player_id = :player_id THEN 1 ELSE 0 END AS
is_white
    FROM games g
    WHERE (g.white_player_id = :player_id OR g.black_player_id =
:player_id)
),
outcomes AS (
    SELECT game_id, time_class,
        CASE
            WHEN (result = '1-0' AND is_white = 1) OR (result = '0-1' AND
is_white = 0) THEN 'win'
            WHEN (result = '0-1' AND is_white = 1) OR (result = '1-0' AND
is_white = 0) THEN 'loss'
            ELSE 'draw'
        END AS outcome
    FROM player_games
),
move_counts AS (
    SELECT m.game_id, COUNT(*) AS cnt
    FROM   moves m
    JOIN   player_games pg ON m.game_id = pg.game_id
    WHERE  (pg.is_white = 1 AND m.color = 'white')
        OR (pg.is_white = 0 AND m.color = 'black')
    GROUP BY m.game_id
)
SELECT
    COUNT(*)                                                AS total_games,
    SUM(CASE WHEN o.outcome = 'win'  THEN 1 ELSE 0 END)  AS wins,
    SUM(CASE WHEN o.outcome = 'loss' THEN 1 ELSE 0 END)  AS losses,
    SUM(CASE WHEN o.outcome = 'draw' THEN 1 ELSE 0 END)  AS draws,
    COALESCE(SUM(mc.cnt), 0)                              AS total_moves
FROM outcomes o
LEFT JOIN move_counts mc ON o.game_id = mc.game_id
```

Clock advantage analysis

Uses four CTEs to compute the average clock difference per game entirely in SQL, then buckets the result with a `CASE WHEN`:

```sql
-- app/crud.py, analyze_clock_advantage()
WITH player_games AS (
    SELECT g.game_id,
           CASE WHEN g.white_player_id = :player_id THEN 'white' ELSE
'black' END AS player_color,
           CASE
               WHEN (g.white_player_id = :player_id AND g.result = '1-0')
                 OR (g.black_player_id = :player_id AND g.result = '0-1')
THEN 'win'
               WHEN g.result = '1/2-1/2' THEN 'draw'
               ELSE 'loss'
           END AS outcome
    FROM games g
    WHERE (g.white_player_id = :player_id OR g.black_player_id =
:player_id)
),
player_clocks AS (
    SELECT m.game_id, m.move_number, m.clock_seconds AS player_clock
    FROM   moves m JOIN player_games pg ON m.game_id = pg.game_id
    WHERE  m.color = pg.player_color AND m.clock_seconds IS NOT NULL
),
opp_clocks AS (
    SELECT m.game_id, m.move_number, m.clock_seconds AS opp_clock
    FROM   moves m JOIN player_games pg ON m.game_id = pg.game_id
    WHERE  m.color != pg.player_color AND m.clock_seconds IS NOT NULL
),
game_advantages AS (
    SELECT pc.game_id, AVG(pc.player_clock - oc.opp_clock) AS avg_advantage
    FROM   player_clocks pc
    JOIN   opp_clocks oc ON pc.game_id = oc.game_id AND pc.move_number =
oc.move_number
    GROUP BY pc.game_id
)
SELECT
    CASE
        WHEN ga.avg_advantage < -30 THEN 'far_behind'
        WHEN ga.avg_advantage < -15 THEN 'behind'
        WHEN ga.avg_advantage <= 15 THEN 'even'
        WHEN ga.avg_advantage <= 30 THEN 'ahead'
        ELSE 'far_ahead'
    END AS clock_bucket,
    pg.outcome
FROM game_advantages ga
JOIN player_games pg ON ga.game_id = pg.game_id
```

Top openings

A straightforward `GROUP BY` with `COUNT(*)` and `ORDER BY`. The function runs this query twice — once filtering on `white_player_id` and once on `black_player_id` — to return openings split by color:

```
-- app/crud.py, get_top_openings()  (run separately for 'white' and
'black')
SELECT   opening_name
FROM     games
WHERE    white_player_id = :player_id   -- swapped to black_player_id for
the second pass
  AND    opening_name IS NOT NULL AND opening_name != ''
  AND    time_class = :time_class
GROUP BY opening_name
ORDER BY COUNT(*) DESC
LIMIT    :limit
```

Part 2: Written Report

2.1 Purpose and Audience

Chess.com provides some built-in statistics, but they are limited in depth and the more detailed ones are locked behind a subscription. This application targets chess players who want a more detailed, personalized breakdown of their game history. Specifically, players who want to understand patterns in their play like when they tend to win, what openings they favor, how they manage time on the clock, and whether their rating trend is improving.

The intended audience is casual to intermediate club-level players who play regularly online and want to evaluate their progress over time.

In my opinion, the most valuable insight is the clock advantage feature. Chess.com doesn't offer any analytics regarding time usage, so I had no way of knowing my own habits before creating this app. I learned that I am habitually behind on the clock compared to my opponents and I win less games because of it. Without this feature, it undoubtedly would have taken me much longer to discover the extent of my time trouble issues.

2.2 App Functionalities

The dashboard provides the a slew of features that draw on clock times, game lengths, win rates, openings, and ratings to generate useful visualizations.

The first feature is rating history, which simply displays a user's elo over a specified date range. I also added a feature to project their rating into the future to estimate how it may change over time.

Next, there are histograms with overlapping line graphs for win rate by game length, clock time advantage, and win rate by opponent rating. These are all stacked histograms that stack wins, losses, and draws in conjunction with line graphs that show the player's win and draw rates at each bin.

For example, in the clock advantage screenshot above, you can see that the largest magnitude of my moves are played with >30 seconds less than my opponent and a corresponding win rate of 47%. When I am more even on time with my opponent, you can see my win rate increases, with a peak of 67% when I am slightly ahead. This tells a player how often they are ahead/behind on the clock and how it's affecting the results of their games.

All these features are filterable by opening choice, including the top 5 most popular openings used by a user, but the wide breadth of openings in chess means a very large amount of data is needed before these graphs populate properly.

In addition to these graphs already mentioned, there is also a time-per-move distribution graph and an average time spent graph that depends on the move number. These numbers aren't very meaningful in isolation, which is why the compare feature is so useful. There is also a recent games feature at the bottom of the dashboard that allows you to reference the game on chess.com.

Screenshots of features not already outlined are shown below.

2.3 Application Hosting

The application runs locally only. As a solo project, there was no infrastructure available for hosting. To run it:

```
uv run uvicorn app.main:app --reload
```

The app is then available at `http://localhost:8000`. Player data is fetched from the Chess.com API on first load and stored in the local SQLite database for all subsequent requests.

2.4 Technical Development and Contributions

All development was done by Joseph Malone.

The project is structured as follows:

Component	Description
`etl/sync_player.py`	On-demand ETL: fetches Chess.com archives, parses PGN in memory, loads into DB
`etl/parse_pgn.py`	PGN parsing utilities (time control classification, clock extraction, opening name)
`etl/fetch_accuracies.py`	Backfills per-game accuracy data from Chess.com for analyzed games

Component	Description
`app/models.py`	SQLAlchemy ORM models for players, games, moves
`app/crud.py`	All analytics queries and data processing
`app/main.py`	FastAPI REST endpoints
`app/static/app.js`	Single-page frontend — Chart.js charts, compare mode, filter controls
`app/static/index.html`	Dashboard layout
`db/schema.sql`	Reference schema with indexes

The backend is built on **FastAPI** and **SQLite**, with all database queries written in SQL executed via `sqlalchemy.text()`. The frontend is vanilla JavaScript with **Chart.js**. Data is sourced exclusively from the **Chess.com public API** — no third-party chess databases are used.

2.5 References

- Chess.com Public API: https://www.chess.com/news/view/published-data-api
- `python-chess` library (PGN parsing): https://python-chess.readthedocs.io
- Chart.js (frontend charting): https://www.chartjs.org
- FastAPI (Python web framework): https://fastapi.tiangolo.com
- SQLAlchemy (ORM): https://www.sqlalchemy.org
- **Claude Code (Anthropic)** — used to help design and implement the frontend JavaScript and HTML once the backend was functional. Also used to debug a bug where the rating history chart showed large instantaneous spikes caused by periods of player inactivity pulling in games from a different time control (fixed via the IQR outlier filter in `elo_history()`). At the end of development, Claude was also used to review the codebase for redundancies and inefficiencies.

2.6 Reflections

What I learned: The most technically interesting part of the backend was writing the more complex analytics queries. For example, the move count query (Section 1.4) requires joining `moves` back to `games` with a conditional filter on player color — a simple `COUNT(*)` would double-count since the `moves` table contains both players' moves. Working through problems like this gave me a much better understanding of SQL intuitively once I had worked with the data for long enough.

Most interesting part: The project directly applies to one of my hobbies--I play chess regularly and now actively use the dashboard to check my own progress. Seeing my rating trend and understanding where I tend to lose time on the clock made the project feel genuinely useful, and I plan on continuing development after the class is over.

Most challenging part: Handling the per-game clock advantage computation (Section 1.4) was the hardest query to get right. Each game requires matching player and opponent moves by move number, computing the per-move clock difference, and averaging all while correctly identifying which color the

player was. Getting the color-filtering logic correct across all analytics features required precision and knowing how the `white_player_id`/`black_player_id` fields interact with the `color` column in `moves`.

What I would change: The most impactful addition would be engine evaluation. I would do this by storing centipawn scores and accuracy per move using a local Stockfish instance. The database already has an `eval_cp` column in the `moves` table reserved for this purpose. With engine data, the app could show blunder tendencies, accuracy trends by opening, and better time-pressure analysis. This would significantly expand what's possible analytically, but I didn't have time to implement it for this project.